

Performance Summary — PairEval

k6 performance suite | target http://localhost:8000 (uvicorn + SQLite) | fixture 100 นักศึกษา / 2,200 pairs | 5 ต.ค. 2026

สิ่งที่ต้องรู้ 4 ข้อ

โหลดระดับวันส่งงานผ่านสบาย

0.00% error

20 VUs, 1,632 requests, 12.4 req/s — threshold ผ่านทั้ง 14 endpoint, checks 100%, ไม่มี 5xx เลย load.js ใช้เป็นด้าน CI ได้

เพดานของงาน aggregate ต่ำกว่าที่คิดมาก

< 2 req/s

1 → 2 req/s ทำให้ p95 โตจาก 539 ms เป็น 11.7 s (22 เท่า จากโหลดเพิ่มเท่าตัว) ที่ 4 req/s ไม่มี request ไหนจบภายใน 120 s

ระบบล้ม ไม่ได้ปฏิเสธ

0 : 208

429/503 ศูนย์รายการ เทียบกับ 5xx/timeout 208 รายการ (stress 46 + breakpoint 162) — ไม่มี rate limit ไม่มีคิว ผู้ใช้ได้หน้าขาวไม่ใช่ “ลองใหม่”

ไม่พบการเสื่อมตามเวลา

+3.0%

soak 3 นาที 3,468 requests — p95 ช่วงท้ายเทียบช่วงต้นต่างกัน 3.0% ถือว่าคงที่ (ยังสั้นเกินกว่าจะสรุปเรื่อง memory leak)

ผลรวมทั้ง 5 scenario

scenario	โหลดที่ใช้	requests	error	5xx/timeout	thresholds	บทบาท
smoke.js	1 VU, 1 iteration ครบทุก journey	56	0.00%	0	PASS	ยืนยัน contract
load.js	ramp 0→20 VUs + อาจารย์ 1 คน	1,632	0.00%	0	PASS	ด้าน CI
soak.js	8 readers + writers 6/นาที, 3 นาที	3,468	0.00%	0	PASS	หา drift
stress.js	ramp ถึง 120 VUs	2,494	1.84%	46	FAIL 14	ดูว่าพังอย่างไร
breakpoint.js	arrival rate 1→5 req/s (ขั้นละ 40s)	764	21.20%	162	FAIL 10	หาเพดาน

stress และ breakpoint แดงโดยเจตนา — ทั้งสองตัวมีหน้าที่ดันระบบให้พังเพื่อดูรูปแบบความล้มเหลว ไม่ใช่ด้านคุณภาพ อย่าเอาไปใส่ CI

load.js — ด้าน CI (20 VUs)

endpoint	tier	budget p95	n	p50	p95	p99	ผล
GET /export/csv	job	30,000	12	159.4	656.8	668.4	PASS
GET /export/xlsx	job	30,000	3	498.9	576.5	583.4	PASS
GET /reports/individual	screen	3,000	7	150.4	292.3	300.3	PASS
GET /reports/coverage	screen	3,000	7	110.0	223.8	241.8	PASS
GET /reports/group	screen	3,000	7	63.3	165.4	172.0	PASS
GET /assignments/{id}/evaluations	screen	3,000	110	56.0	158.1	281.8	PASS
GET /my-score	job	30,000	50	64.0	145.7	204.5	PASS
POST /:recompute	job	30,000	1	102.8	102.8	102.8	PASS
POST /evaluations:submit	screen	3,000	105	20.6	60.7	94.9	PASS
GET /reports/quality	screen	3,000	7	31.3	59.7	67.5	PASS
POST /evaluations/draft (autosave)	interactive	200	1,199	9.3	37.0	55.3	PASS
GET /classrooms/{id}/audit	screen	3,000	7	5.2	33.1	34.2	PASS
POST /auth/google	interactive	200	57	8.8	32.3	44.8	PASS
GET /notifications	interactive	200	50	2.6	5.0	7.6	PASS

SLO แบ่ง 3 tier เพราะหน้าจอเดียวของนักศึกษาเรียก API หลายครั้ง — **interactive** 200 ms (ผู้ใช้รอคาหน้าจอ) / **screen** 3 s (หน้าที่ประกอบจากหลาย query) / **job** 30 s (งาน aggregate) ถ้าใช้เกณฑ์ของ job คลุมทุก endpoint เกณฑ์จะหลวมจนจับ regression ไม่ได้

Hypothesis vs Actual

สมมติฐานทั้ง 4 ข้อถูกบันทึกไว้ในคอมเมนต์ของ k6/lib/journeys.js และ k6/lib/thresholds.js ก่อน การรันครั้งนี้ ไม่ใช่เขียนย้อนหลัง

Hypothesis	ผลจริง	ถูก/ผิด
GET /my-score จะช้าที่สุดในกลุ่มที่นักศึกษาเรียก เพราะรัน scoring engine ใหม่ทั้ง assignment ทุกครั้ง — p95 > 500 ms	p95 = 145.7 ms ที่ 20 VUs	ผิด
GET /export/csv จะเป็น endpoint ที่ช้าที่สุดของทั้งระบบ	p95 = 656.8 ms — ช้าที่สุดจริง	ถูก
autosave POST /evaluations/draft อยู่ใน budget 200 ms ได้แม้ที่ 20 VUs	p95 = 37.0 ms จาก 1,199 requests	ถูก
เพดานของระบบถูกกำหนดโดยงาน aggregate ไม่ใช่จำนวน request	mix หนักตันที่ < 2 req/s ขณะที่ autosave 1,199 requests ผ่านสบาย	ถูก

ข้อที่ 1 ผิด แต่ผิดเรื่อง **scale** ไม่ใช่เรื่องกลไก — ที่ 20 VUs บน SQLite ที่เพิ่งสร้างใหม่ งาน recompute ทั้ง assignment (2,200 comparisons) ยังจบใน ~146 ms แต่ breakpoint test แสดงว่า endpoint เดียวกันไปถึง **120 s (timeout)** เมื่อ arrival rate ขึ้นเป็น 2–3 req/s สมมติฐานถูกเรื่องกลไก แต่ผิดเรื่องจุดที่กลไกนั้นเริ่มเจ็บ — นี่คือเหตุผลที่ load test ตัวเดียวไม่พอ

Bottleneck

1. scoring engine รันใหม่ทั้ง assignment ในทุก request ที่เกี่ยวกับคะแนน

หลักฐานจากโค้ด — backend/app/features/scoring/services.py:22 run_scoring_engine_for_assignment() โหลด ทุก PairAssignment (2,200 แถว) และ ทุก Comparison ของ assignment เข้าหน่วยความจำแล้วคำนวณคะแนนใหม่หมด ไม่มี cache และไม่อ่านจากตารางคะแนนที่ persist ไว้ ถูกเรียกจาก 6 จุด:

ที่เรียก	endpoint	ความถี่ที่เกิดจริง
scoring/services.py:318 get_student_score_view	GET /my-score	ทุกครั้งที่นักศึกษา 1 คนเปิดดูคะแนนตัวเอง
scoring/services.py:174, 195, 274	:recompute, score views	อาจารย์กด
reporting/services.py:19, 79	/reports/*, /export/*	อาจารย์เปิดหน้ารายงาน

ต้นทุนต่อ 1 request เป็น O(จำนวน comparison ทั้ง assignment) — ไม่ขึ้นกับว่าใครถาม ห้องเรียน 100 คนเปิดดูคะแนนตัวเองพร้อมกัน = recompute ทั้ง assignment 100 รอบ เพื่อตอบคำถามที่แต่ละคนต้องการแค่ 2 ตัวเลขของตัวเอง

หลักฐานจากการวัด — breakpoint ไล่ arrival rate ของ mix หนัก (:recompute + /reports/individual + /export/csv + /my-score):

เป้า req/s	n (สำเร็จ)	p50	p95	อ่านว่าอะไร
1	164	99.7 ms	538.8 ms	ยังไหว
2	252	3,701 ms	11,747 ms	คิวเริ่มต้น — p95 โต 22 เท่า
3	174	10,680 ms	69,294 ms	ตันแล้ว
4	0	—	—	ไม่มี request ไหนจบใน 120 s
5	0	—	—	ไม่มี request ไหนจบใน 120 s

การโตจาก 539 ms เป็น 11.7 s เมื่อโหลดเพิ่มแค่เท่าตัว คือรูปร่างของคิวที่ตันแล้ว ไม่ใช่การช้าลงแบบเป็นเส้นตรง เพดานของระบบในการตั้งค่านี้อยู่ระหว่าง **1–2 req/s** ของงาน aggregate (295 iterations ถูก k6 หักเพราะระบบรับ rate ที่ตั้งไว้ไม่ทัน)

2. ไม่มี load shedding — ระบบล้นแทนที่จะปฏิเสธ

การรัน	429/503 (ปฏิเสธอย่างสุภาพ)	5xx/timeout (ล้น)
stress (120 VUs)	0	46
breakpoint (ถึง 5 req/s)	0	162

ไม่มี rate limit ไม่มีคิว ไม่มี timeout ฝั่งแอป — พอเกินกำลัง ผู้ใช้ได้ 500 หรือค้างจนหมดเวลา ไม่ได้ข้อความ “ลองใหม่อีกครั้ง” ซึ่งเป็นสิ่งที่คู่กันได้ ต่างกันทั้งในแง่ UX และความสามารถในการกู้ระบบ

3. SQLite serialize การเขียน

ใน stress test submit_eval มี p95 = 3.2 s แต่ p99 = **59.1 s** — ช่องว่างแบบนี้ (p99 มากกว่า p95 เกือบ 20 เท่า) คือลายเซ็นของการรอคิวล็อก ไม่ใช่ของงานที่หนักขึ้นสม่ำเสมอ ข้อนี้เป็นข้อจำกัดของสภาพแวดล้อมทดสอบ ยังสรุปเป็น bottleneck ของ production ไม่ได้ ต้องวัดซ้ำบน Postgres ก่อน

สิ่งที่จะแก้ (ยังไม่แก้ในวันนี้)

ลำดับ	แนวทาง	คาดว่าจะได้อะไร
1	persist ผลของ scoring engine ลงตารางคะแนน + invalidate เมื่อมี comparison ใหม่ แล้วให้ GET /my-score อ่านจากตารางนั้น	p95 ที่ load scale: 146 ms → < 30 ms ; ที่ 2 req/s: 11.7 s → ระดับ interactive เพราะต้นทุนต่อ request เปลี่ยนจาก O(ทั้ง assignment) เป็น O(1)
2	ให้ get_student_score_view ดึงเฉพาะคะแนนของกลุ่มและของผู้เรียกที่ร้องขอ ไม่ต้องคำนวณทั้ง assignment	ลดงานต่อ request ลง ~100 เท่าในห้อง 100 คน
3	ใส่ rate limit + timeout ฝั่งแอปบน endpoint tier job ให้ตอบ 429 เมื่อคิวเต็ม	ย้ายตัวเลข 208 รายการจากคอลัมน์ “ลั้ม” ไปคอลัมน์ “ปฏิเสธอย่างสุภาพ” — pe_hard_failures กลับเป็น 0
4	ใช้ Postgres บน staging ไม่ใช่ SQLite	ปิดช่องว่าง p95/p99 ของ endpoint ที่เขียน (submit_eval p99 59 s)

ของที่ส่งมอบ และวิธีรันซ้ำ

ไฟล์	คืออะไร
k6/smoke.js	1 VU 1 iteration ครบทุก journey — ยืนยันว่าสคริปต์ยังติดก่อนเสียเวลารันตัวอื่น
k6/load.js	ด้าน pass/fail ของ CI — นักศึกษา + อาจารย์พร้อมกัน
k6/stress.js	ต้นเกินกำลัง ดูรูปแบบความลั้มเหลว (429/503 เทียบ 500/timeout)
k6/soak.js	โหลดปกติลากยาว จับ drift ของ p95 ด้วย tag early/mid/late
k6/breakpoint.js	ไล่ arrival rate เป็นขั้น หาเพดาน req/s
k6/lib/ (6 ไฟล์)	harness: config / API client / fixture / journeys / thresholds / summary
k6/results/	*-summary.json (ผลดิบ) + *-report.md (ตารางอ่านง่าย) ของทั้ง 5 scenario
k6/baseline/	baseline ของ load / smoke / soak — รอบถัดไปรายงานจะขึ้นธง REGRESSED ให้เอง
k6/README.md	วิธีรัน, env ทั้งหมด, ตัวอย่าง CI job
docs/performance-report.md	รายงานเต็มพร้อม AI analysis

รันซ้ำ (ต้องอยู่ใน k6/ เพราะ handleSummary เขียนลง ./results)

```
DATABASE_URL="sqlite:///./k6-perf.db" python -u -m uvicorn backend.app.main:app --port 8000

cd k6
k6 run smoke.js          # ยืนยันว่ายังติดก่อน
k6 run load.js           # ด้าน pass/fail - exit 0 = ผ่าน
k6 run stress.js         # ดูว่าพังอย่างไร
k6 run breakpoint.js     # หาเพดาน req/s
PE_SOAK_DURATION=30m k6 run soak.js
```

ยืนยันแล้วว่า gate ทำงานจริง — PE_SLO_INTERACTIVE_MS=1 k6 run smoke.js คืน **exit code 99** ซึ่งทำให้ CI แดงตามที่ควร

ยังไม่ได้ทำ

- 1. **Structured logging** (ใบงานชิ้นที่ 2) — ฝั่ง backend ยังไม่มี requestLogger, correlation id ใน log, และการ redact email/token/password — k6 ส่ง X-Request-Id ให้ทุก request อยู่แล้ว รอฝั่ง backend echo ลง log
- 2. **CI job** (ใบงานชิ้นที่ 3) — repo นี้ยังไม่มี .github/workflows/ เลย YAML ของ job performance เขียนไว้ให้ใน k6/README.md ก็อปไปวางได้
- 3. **soak ยาว 30 นาที** พร้อมเฝ้า RSS ฝั่งเซิร์ฟเวอร์ ก่อนจะสรุปเรื่อง memory leak ได้จริง

PairEval performance suite — k6 v2.2.0 · ทุกการรันยิงใส่ localhost เท่านั้น (assertSafeTarget() ปฏิเสธ target ที่ไม่ใช่ localhost เว้นแต่ตั้ง PE_ALLOW_REMOTE=yes) · รายละเอียดเต็มใน docs/performance-report.md